Lab 3: Memory Allocator 實驗室三：記憶體分配器

Contents

- Introduction
- Goals of this lab
- Background
- Basic Exercises
- Advanced Exercises

Introduction 簡介#

A kernel allocates physical memory for maintaining its internal states and user programs' use. Without memory allocators, you need to statically partition the physical memory into several memory pools for different objects. It's sufficient for some systems that run known applications on known devices. Yet, general-purpose operating systems that run diverse applications on diverse devices determine the use and amount of physical memory at runtime. On OrangePi RV2, available physical memory and reserved regions are described by the Device Tree, so you must read those descriptions at boot.

核心分配實體記憶體以維持其內部狀態及使用者程式的使用。沒有記憶體分配器，你需要靜態將實體記憶體分割成多個不同的物件的記憶體池。對於某些在已知裝置上執行已知應用程式的系統來說，這已經足夠了。然而，執行不同裝置上各種應用程式的通用作業系統，決定了執行時物理記憶體的使用量與使用量。在 OrangePi RV2 上，可用的實體記憶體和預留區域會由裝置樹描述，因此你必須在開機時閱讀這些描述。

In Lab 3, you need to implement memory allocators that rely on OrangePi RV2's Device Tree for memory layout. They'll be used in all later labs.

在 Lab 3，你需要實作依賴 OrangePi RV2 裝置樹來配置記憶體的記憶體配置器。它們會在所有後續實驗室使用。

Goals of this lab 本實驗室目標#

- Implement a **Page Frame Allocator**.

  實作頁面框架分配器。

- Implement a **Dynamic Memory Allocator** that requests pages from the previous Page Frame Allocator and partitions pages into chunk pools.

  實作動態記憶體分配器，從前一個頁面框架分配器請求頁面，並將頁面分割成區塊池。

- Implement a **Startup Allocator** (bump-pointer).

  實作啟動配置器（bump-pointer）。

Background 背景#

Reserved Memory 保留記憶體#

After OrangePi RV2 is booted, some physical memory is already in use. For example:

當 OrangePi RV2 啟動後，部分實體記憶體已經被使用中。例如：

1. Device Tree Blob (DTB) 裝置樹狀物（DTB）
2. Kernel image 核心映像檔
3. Initramfs (ramdisk) Initramfs （ramdisk）
4. Any additional platform-specific reserved areas (e.g., spin tables, CLINT/PLIC buffers)

   任何額外的平台專用保留區（例如自旋表、CLINT/PLIC 緩衝區）

Your memory allocator should not allocate these memory blocks. Thus, you need to reserve these memory blocks before initializing any allocator.

你的記憶體配置器不應該分配這些記憶體區塊。因此，你需要在初始化任何配置器前先保留這些記憶體區塊。

Dynamic Memory Allocator

動態記憶體分配器#

Given the allocation size, a dynamic memory allocator needs to find a large enough contiguous memory block and return the pointer to it. Also, the memory block would be released after use. A dynamic memory allocator should be able to reuse the memory block for another memory allocation. Reserved or allocated memory should not be allocated again to prevent data corruption.

根據分配大小，動態記憶體配置器需要找到足夠大的連續記憶體區塊並回傳指標。此外，使用後記憶體區塊會被釋放。動態記憶體分配器應該能將該記憶體區塊用於另一個記憶體配置。保留或分配記憶體不應再次分配，以防止資料損壞。

Page Frame Allocator 頁框分配器#

You may be wondering why you are asked to implement another memory allocator for page frames. Isn't a well-designed dynamic memory allocator enough for all dynamic memory allocation cases? Indeed, you don't need it if you run applications in kernel space only.

你可能會好奇為什麼要為頁面框架實作另一個記憶體分配器。設計良好的動態記憶體分配器還不夠用來處理所有動態記憶體分配的情況嗎？事實上，如果你只在核心空間執行應用程式，其實不需要它。

However, if you need to run user space applications with virtual memory, you'll need a lot of 4KB memory blocks with 4KB memory alignment called page frames. That's because 4KB is the unit of virtual memory mapping. Regular dynamic memory allocators that store metadata (like size headers) within the memory block are unsuitable for managing page frames. Since page frames must be strictly 4KB-aligned, an inline header would shift the page boundary, forcing the allocator to waste significant space to maintain alignment.

不過，如果你需要用虛擬記憶體執行使用者空間應用程式，就需要大量 4KB 記憶體區塊，並搭配 4KB 記憶體對齊，稱為頁面框架。因為 4KB 是虛擬記憶體映射的單位。一般的動態記憶體分配器會在記憶體區塊中儲存元資料（如大小標頭）並不適合管理頁面框架。由於頁面框架必須嚴格符合 4KB 對齊，內嵌標頭會移動頁面邊界，迫使配置器浪費大量空間來維持對齊。

Therefore, a better approach is representing available physical memory as page frames. The page frame allocator reserves and uses an additional page frame array to bookkeep the use of page frames. The page frame allocator should be able to allocate contiguous page frames for allocating large buffers. For fine-grained memory allocation, a dynamic memory allocator

can allocate a page frame first then cut it into chunks.

因此，更好的方法是將可用實體記憶體表示為頁面框架。頁框配置器保留並使用額外的頁框陣列來記錄頁框的使用。頁面框架分配器應該能夠分配連續的頁面框架以分配大型緩衝區。對於細粒度記憶體配置，動態記憶體配置器可以先分配一個頁面框架，然後再將其切割成區塊。

Observability of Allocators 配置器的可觀察性#

It's hard to observe the internal state of a memory allocator and hence hard to demo. To check the correctness of your allocator, you need to **print the log of each allocation and free**. Please refer to How to Demo? for more information.

很難觀察記憶體分配器的內部狀態，因此也很難進行示範。要檢查分配器的正確性，你需要列印每次分配的記錄。請參考「如何示範？」一文。欲了解更多資訊。

Basic Exercises 基本練習#

In the basic part, your allocator doesn't need to deal with the reserved memory problem. You can find an unused memory region (e.g. `0x1000_0000` - `0x2000_0000`) and manage that part of memory only.

基本上，你的配置器不需要處理保留記憶體的問題。你可以找到一個未使用的記憶體區域（例如 `0x1000_0000` - `0x2000_0000` ），並只管理那部分記憶體。

Note 註

In Advanced Exercise 2 - Reserved Memory - 10%, your allocator will transition to fully dynamic initialization. You will be required to use the memory regions from the Device Tree's `<memory>` node instead of hardcoding a region.

在進階練習 2 - 保留記憶體 - 10% 中，你的配置器將轉換成完全動態初始化。你必須使用裝置樹 `<memory>` 節點的記憶體區域，而不是硬編碼區域。

The major reason for hardcoding early on is that it saves you from handling the reserved memory problem right now. If you dynamically managed the entire Device Tree memory space in your basic setup, your allocator might accidentally allocate critical memory that is already in use by the system (e.g., `0x0000_0000` - `0x0007_ffff`). By picking a safe, hardcoded region, you ensure these reserved regions are avoided.

早期硬編碼的主要原因是這樣可以省去你現在處理預留記憶體的問題。如果你在基本設定中動態管理整個裝置樹記憶體空間，配置器可能會意外分配系統已在使用的關鍵記憶體（例如，`0x0000_0000` - `0x0007_ffff`）。選擇一個安全且硬編碼的區域，可以確保避免這些保留區域。

Basic Exercise 1 - Buddy System - 40%
基本練習 1 - 夥伴系統 - 40%#

Buddy system is a well-known and simple algorithm for allocating contiguous memory blocks. It has an internal fragmentation problem, but it's still suitable for page frame allocation because the problem can be reduced with the dynamic memory allocator. We provide one possible implementation in the following part. You can still design it yourself as long as you follow the specification of the buddy system.

夥伴系統是一種廣為人知且簡單的連續記憶體區塊分配演算法。它內部有碎片問題，但仍適合用於頁面框架分配，因為這個問題可以用動態記憶體分配器來減少。接下來的部分將提供一種可能的實作方式。只要遵守夥伴系統的規範，你仍然可以自己設計。

Todo 全部

Implement the buddy system for contiguous page frame allocation. Your maximum order must be larger than 5.

實作夥伴系統以實現連續的頁面框架分配。你的最高訂單必須超過 5。

Note 註

You don't need to handle the case of out-of-memory.

你不需要處理記憶體不足的情況。

Hint 提示

Implementing a generic circular doubly linked list may be very helpful in this lab and in future labs (Reference).

實作通用的循環雙鏈結串列在本實驗室及未來實驗室中可能非常有幫助（參考文獻）。

Data Structure 資料結構#

The Frame Array (or "*The Array*", so to speak)

框架陣列（或可稱為「陣列」）

The Array represents the allocation status of the memory by constructing a 1-1 relationship between the physical memory frame and *The Array*'s entries. For example, if the size of the total allocable memory is 200 KiB with each frame being 4 KiB. Then *The Array* would consist of 50 entries. Use the base addresses from the `<memory>` regions to compute each frame's physical address (e.g., if the first region begins at `0x8000_0000`, its first entry represents `0x8000_0000`, next `0x8000_1000`, etc.).

陣列透過在實體記憶體框架與陣列的條目之間建立 1：1 的關係來表示記憶體的配置狀態。例如，若可分配記憶體的總大小為 200 KiB，且每個框架為 4 KiB。然後陣列會包含 50 個參賽項目。利用 `<memory>` 區域的基底位址來計算每個框架的物理位址（例如，如果第一個區域從開始 `0x8000_0000` ，其第一個項目代表 `0x8000_0000` 、 、 `0x8000_1000` ，等等）。

However, to describe a living Buddy system with *The Array*, we need to provide extra meaning to items in *The Array* by assigning values to them, defined as followed:

然而，要描述一個帶有陣列的活夥伴系統，我們需要透過指派值來賦予陣列中的項目額外的意義，定義如下：

For each entry in *The Array* with index idx and value val (Suppose the framesize to be `4kb`)

對於陣列中每個索引 idx 為 和 值 val 的項目（假設框架大小為 `4kb` ）

- if $\mathrm{val} \geq 0$: 若 $\mathrm{val} \geq 0$ ：

  There is an allocable, contiguous memory that starts from the idx'th frame with $\mathrm{size} = 2^{\mathrm{val}} \times$ `4kb`.

  存在一個可連貫的記憶，從 idx 第個框架開始。 $\mathrm{size} = 2^{\mathrm{val}} \times$ `4kb`

- if $\mathrm{val} = <\mathrm{F}>$: (user-defined value)

  如果 $\mathrm{val} = <\mathrm{F}>$ ：（使用者定義值）

  The idx'th frame is free, but it belongs to a larger contiguous memory block. Hence, buddy system doesn't directly allocate it.

  idx 第一個框架是自由的，但它屬於一個更大的連續記憶區塊。因此，夥伴系統不會直接分配這些資源。

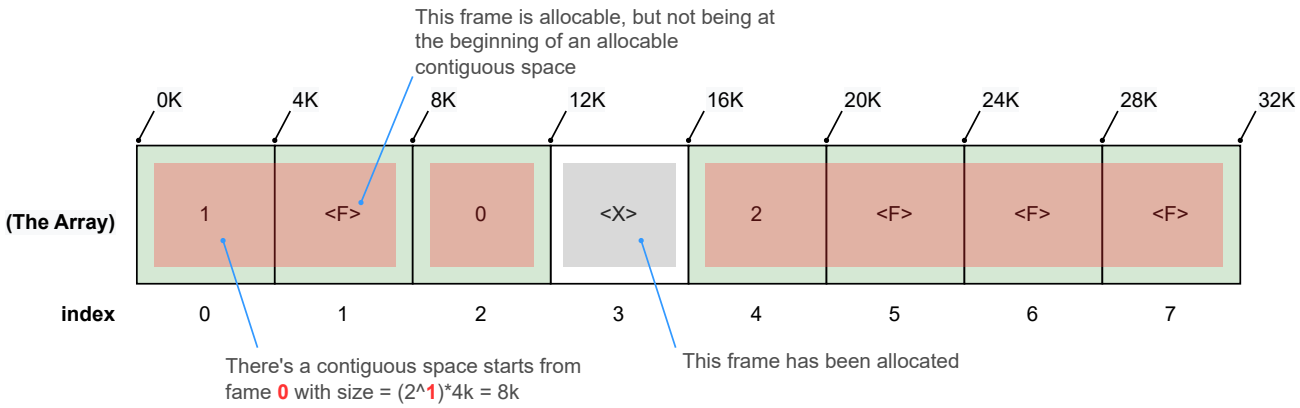
- if $\mathrm{val} = <\mathrm{X}>$: (user-defined value)

  如果 $\mathrm{val} = <\mathrm{X}>$ ：（使用者定義值）

  The idx'th frame is already allocated, hence not allocable.

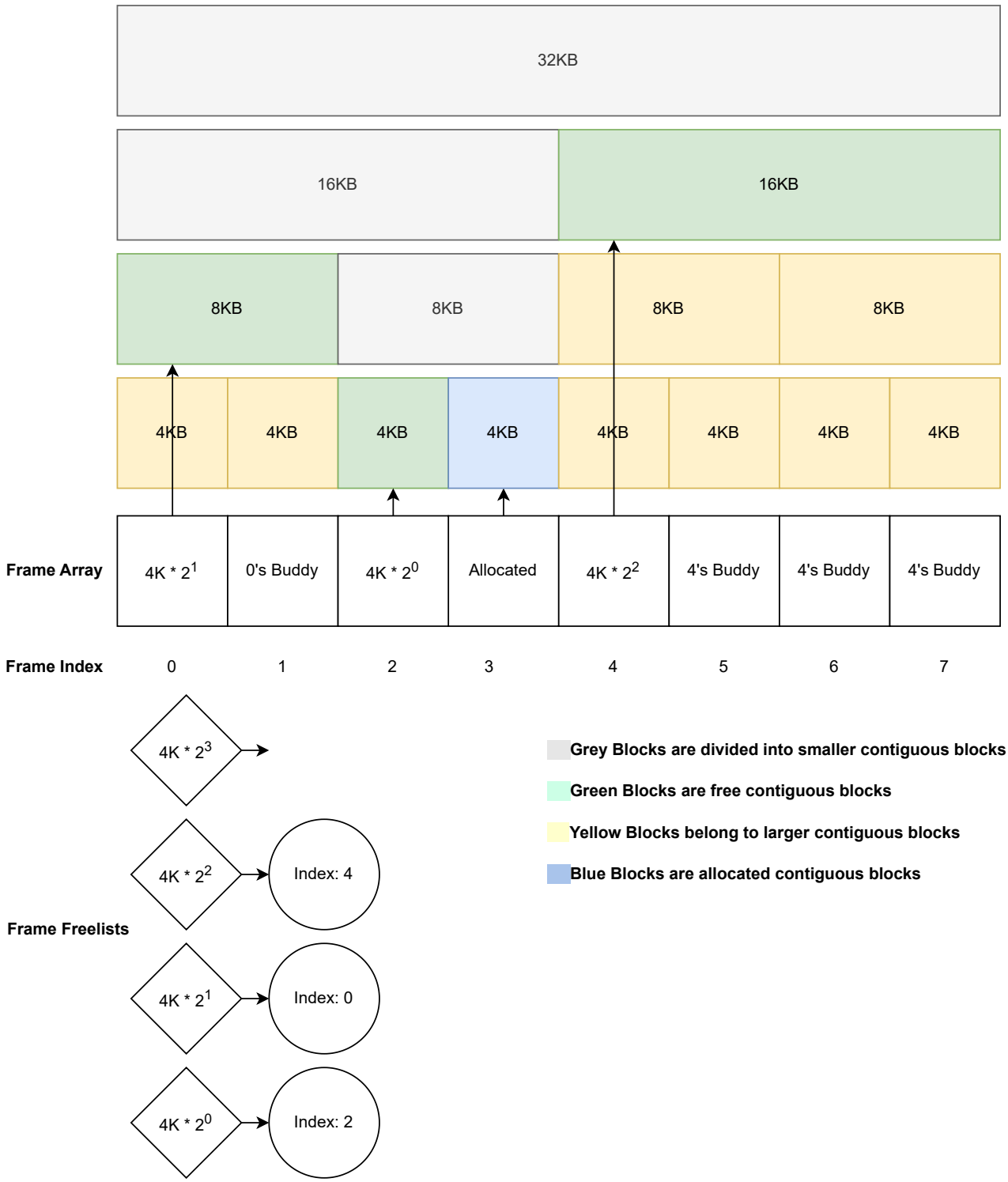
  idx 第個框架已經被分配，因此不可配。

Meaning of the Frame Array



Below is the generalized view of **The Frame Array**:

以下是框架陣列的概括視角：



You can calculate the address and the size of the contiguous block by the following formula.

你可以用以下公式計算連續區塊的地址和大小。

- $\text{block's physical address} = \text{block's index} \times 4096 + \text{base address}$
- $\text{block's size} = 4096 \times 2^{\text{block's exponent}}$

Linked-lists for free blocks with different size (Free List)
不同大小的免費區塊連結清單（Free List）#

You can set a maximum contiguous block size and create one linked-list for each size. The linked-list links free blocks of the same size. The buddy allocator's search starts from the specified block size list. If the list is empty, it tries to find a larger block in a larger block list.

你可以設定最大連續區塊大小，並為每個大小建立一個連結清單。連結列表會釋放相同大小的區塊。夥伴分配者的搜尋從指定的區塊大小清單開始。如果清單是空的，它會嘗試在更大的方塊清單中尋找更大的區塊。

Release redundant memory block
釋放冗餘記憶體區塊#

The above algorithm may allocate one block far larger than the required size. The allocator should cut off the bottom half of the block and put it back to the buddy system until the size equals the required size.

上述演算法可能會分配一個遠大於所需大小的區塊。分配者應該將區塊的下半部切掉，並放回夥伴系統，直到大小等於所需大小。

> **Note 註**
>
> You should print the log of releasing redundant memory block for the demo
>
> 你應該列印示範版中釋放冗餘記憶體區塊的日誌

Free and Coalesce Blocks
自由區塊與聚合區塊#

To allow the buddy system to reconstruct larger contiguous memory blocks, when the user frees an allocated block, the buddy allocator should not naively place it back on the free list. It should try to Find the buddy and Merge iteratively.

為了讓夥伴系統重建更大的連續記憶體區塊，當使用者釋放已分配的區塊時，夥伴分配者不應天真地將該區塊放回空缺清單。它應該嘗試以迭代方式尋找夥伴並合併。

Find the buddy 找到夥伴#

You can use the block's index XORed with its size in pages (i.e., `1 << exponent`) to find its buddy. If its buddy is in the page frame array, then you can merge them into a larger block.

你可以用區塊的索引與其大小（以頁數）XOR 結合（即 `1 << exponent` ）來找到它的夥伴。如果它的夥伴在頁面框架陣列中，你可以將它們合併成更大的區塊。

Merge iteratively 迭代合併#

There is still a possible buddy for the merged block. You should use the same way to find the buddy of the merged block. When you can't find the buddy of the merged block or the merged block size is maximum-block-size, the allocator stops and puts the merged block onto the free list.

合併區塊仍有可能的夥伴。你應該用同樣的方法來找到合併區塊的夥伴。當你找不到合併區塊的夥伴或合併區塊大小為最大區塊大小時，分配者會停止並將合併區塊放入自由清單。

> **Note 註**
>
> You should print the log of merge iteration for the demo.
>
> 你應該列印示範版的合併迭代日誌。

Basic Exercise 2 - Dynamic Memory Allocator - 30% 基本練習二 - 動態記憶分配器 - 30%#

Your Page Frame Allocator (from Basic Exercise 1) provides 4 KB-aligned page frames. The Dynamic Memory Allocator must request page frames from the Page Frame Allocator and use their physical base address. For small allocations (< 4 KB), you may maintain multiple chunk pools (sizes 16, 32, 48, 96 bytes, etc.), partitioning each page into fixed-size chunks for the corresponding pool.

你的頁面框架分配器（來自基本練習 1）提供 4 個 KB 對齊的頁面框架。動態記憶體分配器必須向頁框分配器請求頁面框架，並使用其實體基址。對於小分配（< 4 KB），你可以維護多個區塊池（大小為 16、32、48、96 位元組等），並將每個頁面分割成固定大小的區塊，以對應的池為單位。

On each allocation request:

在每次分配申請中：

1. Round up the requested size to the nearest pool size.

   將要求的水池大小四捨五入到最接近的水池大小。

2. If a free chunk exists in the corresponding pool, return it; otherwise, request a new page from the Page Frame Allocator.

   如果對應池中有空餘區塊，則回傳該區塊;否則，請向頁面框架分配器申請新頁面。

3. Slice the new page frame into chunks and add them to the pool's free list, then return one chunk.

   將新頁面框架切成區塊，加入池的自由清單，然後回傳一個區塊。

When freeing a chunk, use its base page frame address to identify which pool it belongs to, and place it back onto that pool's free list.

在釋放區塊時，利用其基底頁面框架位址來識別它屬於哪個池，然後將其放回該池的空間清單。

> **Todo 全部**
>
> Implement a dynamic memory allocator.
>
> 實作動態記憶體分配器。

How to Demo? 如何拆除？#

Run test case 執行測試案例#

TAs will provide test cases during the demo, your implementation should expose two APIs `allocate` and `free` (can be renamed).

TA 會在示範時提供測試案例，你的實作應該會暴露兩個 API， `allocate` 並且 `free` （可以重新命名）。

- `allocate(size)`: Allocates a memory block of size bytes and returns a pointer to the starting address of that memory segment.

  `allocate(size)` 分配一個大小為位元組的記憶體區塊，並回傳指向該記憶體區段起始位址的指標。

- `free(ptr)`: Frees the memory previously allocated. ptr is the pointer returned by allocate.

`free(ptr)` ：釋放先前分配的記憶體。PTR 是 Allocate 回傳的指標。

Constraints: 限制條件：

1. 1 ≤ `size` ≤ 2,147,483,647
2. `ptr` is a pointer returned by `allocate`.

   `ptr` 是由 `allocate` 返回的指標。

Here is an example of the test case.

這裡有一個測試案例的範例。

```c
void test_alloc_1() {
    uart_puts("Testing memory allocation...\n");
    char *ptr1 = (char *)allocate(4000);
    char *ptr2 = (char *)allocate(8000);
    char *ptr3 = (char *)allocate(4000);
    char *ptr4 = (char *)allocate(4000);

    free(ptr1);
    free(ptr2);
    free(ptr3);
    free(ptr4);

    /* Test kmalloc */
    uart_puts("Testing dynamic allocator...\n");
    char *kmem_ptr1 = (char *)allocate(16);
    char *kmem_ptr2 = (char *)allocate(32);
    char *kmem_ptr3 = (char *)allocate(64);
    char *kmem_ptr4 = (char *)allocate(128);

    free(kmem_ptr1);
    free(kmem_ptr2);
    free(kmem_ptr3);
    free(kmem_ptr4);

    char *kmem_ptr5 = (char *)allocate(16);
    char *kmem_ptr6 = (char *)allocate(32);

    free(kmem_ptr5);
    free(kmem_ptr6);

    // Test allocate new page if the cache is not enough
    void *kmem_ptr[102];
    for (int i=0; i<100; i++) {
        kmem_ptr[i] = (char *)allocate(128);
    }
    for (int i=0; i<100; i++) {
        free(kmem_ptr[i]);
    }

    // Test exceeding the maximum size
    char *kmem_ptr7 = (char *)allocate(MAX_ALLOC_SIZE + 1);
    if (kmem_ptr7 == NULL) {
        uart_puts("Allocation failed as expected for size > MAX_ALLOC_SIZE\n");
    }
    else {
        uart_puts("Unexpected allocation success for size > MAX_ALLOC_SIZE\n");
        free(kmem_ptr7);
    }
}
```

Note 註

You need to copy the test case to your kernel during demo. The variables and functions can be renamed; you can modify the test case to satisfy your needs.

你需要在示範時把測試案例複製到核心。變數與函式可以重新命名;你可以修改測試案例以符合需求。

Print log 列印日誌#

Since the design of the memory allocator can vary, it's your responsibility to explain your design to TAs and prove its correctness through the log during the demo. You can design the log format yourself, but make sure that it is sufficient for your explanation. Here is a simple example for your reference:

由於記憶體分配器的設計可能有所不同，你有責任向助教說明你的設計，並在示範時透過日誌證明其正確性。你可以自己設計日誌格式，但務必確保它足夠用於你的說明。這裡有一個簡單的例子供參考：

```
When
1. Adding a block to the free list:
    [+] Add page 1 to order 0. Range of pages: [1, 1]
2. Removing a block from the free list:
    [-] Remove page 4 from order 2. Range of pages: [4, 7]
3. Finding a buddy:
    [*] Buddy found! buddy idx: 16898 for page 16899 with order 0
4. Allocating a page:
    [Page] Allocate 0x4000 at order 1, page 4. Next address at order 1: 0x6000
5. Freeing a page:
    [Page] Free 0x4000 and add back to order 2, page 4. Next address at order 2: 0x
6. Allocating a chunk:
    [Chunk] Allocate 0x3030 at chunk size 16
7. Freeing a chunk:
    [Chunk] Free 0x3030 at chunk size 16
8. Reserving memory (In advance part):
    [Reserve] Reserve address [0x0, 0x1000). Range of pages: [0, 1)
```

Todo 全部

Print the log of the memory allocator.

列印記憶體分配器的日誌。

Advanced Exercises 進階練習#

Advanced Exercise 1 - Efficient Page Allocation - 10%
進階練習一 - 有效頁面分配 - 10%#

Basically, when you dynamically assign or free a page, your buddy system's response time should be as quick as possible. In the basic part, we only care about the correctness of your implementation, but in this section, you must devise a technique to access the page frame node in constant time.

基本上，當你動態指派或釋放頁面時，你的夥伴系統的回應時間應該盡可能快。在基本階段，我們只關心你實作的正確性，但在本節中，你必須設計一種技巧，以恆定時間存取頁面框架節點。

> **Todo 全部**
>
> You should allocate and free a page in O(log n), while ensuring any page frame lookup is O(1).
>
> 你應該分配並釋放一個頁面，範圍為 O（log n），同時確保任何頁面框架查詢為 O（1）。

Advanced Exercise 2 - Reserved Memory - 10%
進階練習二 - 保留記憶 - 10%#

In the basic exercises, you hardcoded a specific allocable memory region. Now, you need to transition to a dynamic memory layout by finding the actual allocable physical memory from the Device Tree. Since you have already implemented the Device Tree parser in Lab 2, you can directly reuse your implementation to parse the `<memory>` node and get the available memory regions.

在基本練習中，你要硬編碼一個特定的可分配記憶區域。現在，你需要透過從裝置樹中找到實際可分配的實體記憶體，來轉換到動態記憶體配置。既然你已經在 Lab 2 實作了裝置樹解析器，你可以直接重複使用你的實作來解析 `<memory>` 節點並取得可用的記憶體區域。

Note 註

There may be many memory regions in the Device Tree. For example, OrangePi RV2 has two memory regions:

裝置樹中可能有許多記憶體區域。例如，OrangePi RV2 有兩個記憶體區域：

- `0x0000_0000_0000_0000` - `0x0000_0000_7FFF_FFFF` (2 GiB)

  `0x0000_0000_0000_0000` - `0x0000_0000_7FFF_FFFF` （2 GiB）
- `0x0000_0001_0000_0000` - `0x0000_0001_7FFF_FFFF` (2 GiB)

  `0x0000_0001_0000_0000` - `0x0000_0001_7FFF_FFFF` （2 GiB）

You don't have to handle all memory regions. You may use only the first one.

你不必處理所有記憶體區域。你只能使用第一個。

Furthermore, as previously noted in the background, when OrangePi RV2 is booted, specific memory regions must be reserved to prevent data corruption. Use the values you parse directly to reserve:

此外，如前所述，當 OrangePi RV2 啟動時，必須保留特定的記憶體區域以防止資料損毀。直接用你解析的值來保留：

1. DTB Blob: use the `fdt_ptr` and `fdt_header.totalsize` to obtain start and size.

   DTB Blob：使用 `fdt_ptr` and `fdt_header.totalsize` 來取得起始和大小。
2. Kernel Image: obtain its physical memory boundaries via symbols defined in the linker script.

   核心映像：透過連結腳本中定義的符號取得其實體記憶體邊界。
3. Initramfs: use `/chosen` properties `linux,initrd-start` and `linux,initrd-end` to obtain its range. (already done in Lab 2)

   初始化：利用 `/chosen` 屬性 `linux,initrd-start` 和 `linux,initrd-end` 取得其射程。（實驗室 2 已經完成了）
4. Any additional reserved regions (if have, e.g., spin tables, DMA buffers, platform-specific). You may retrieve via `/reserved-memory` node in the device tree.

   任何額外的保留區域（例如有自旋表、DMA 緩衝區、平台特定區域）。你可以透過 `/reserved-memory` 裝置樹中的節點來取得資料。

The following code is a brief example, you may design your own:

以下程式碼是一個簡短範例，你可以自行設計：

```
void memory_reserve(uint64_t start, uint64_t size) {
  // Mark all 4 KB page frames in [start, start + size) as reserved
}
```

Todo 全部

- Retrieve the memory regions from the Device Tree `<memory>` node to replace the hardcoded region. You can directly use the parsing functions implemented in Lab 2.

  從裝置樹 `<memory>` 節點擷取記憶體區域，以取代硬編碼的區域。你可以直接使用 Lab 2 裡實作的解析函數。

- Design an API to reserve memory blocks. Use the values obtained to call it for each of the four categories above.

  設計一個 API 來保留記憶體區塊。利用取得的數值來判定上述四個類別中的每一個。

Advanced Exercise 3 - Startup Allocation - 20%
進階練習三 - 啟動分配 - 20%#

In general purpose operating systems, the amount of physical memory is determined at runtime. Hence, a kernel needs to dynamically allocate its page frame array for its page frame allocator (e.g., *The Frame Array*). The page frame allocator then depends on dynamic memory allocation. The dynamic memory allocator depends on the page frame allocator. This introduces the chicken or the egg problem. To break the dilemma, you need a dedicated startup allocator during startup time.

在通用作業系統中，實體記憶體的數量是在執行時決定的。因此，核心需要動態地為其頁框配置器（例如框架陣列）分配頁面框架陣列。頁框分配器則依賴動態記憶體配置。動態記憶體分配器依賴於頁面框架分配器。這就引發了先有雞還是先有蛋的問題。要解決這個困境，你需要在啟動期間有專屬的創業分配員。

The design of the startup allocator is quite simple. Implement a minimal bump-pointer allocator at startup that does not rely on the Page Frame Allocator.

新創公司分配器的設計相當簡單。在啟動時實作一個最小的凸點指標分配器，且不依賴頁面框架分配器。

This startup allocator must:

此新創配置人必須：

1. Before allocating *The Frame Array* or other structures, mark reserved regions such as DTB blob, kernel image, etc.

   在分配 Frame Array 或其他結構前，先標記保留區域，如 DTB blob、kernel image 等。

2. From the first usable memory region after those reservations, allocate a contiguous region (4KB-aligned) of length `frame_array_size = total_page_count × sizeof(struct frame_struct)`.

   從這些保留後的第一個可用記憶體區域起，分配一個連續（4KB 對齊）區域 `frame_array_size = total_page_count × sizeof(struct frame_struct)` ，長度為 。

3. Initialize the Page Frame Array data structures in that region.

   在該區域初始化頁面框架陣列的資料結構。

4. Handover control to the Buddy System, which will manage all remaining free 4 KB page frames.

   切換控制權交給 Buddy 系統，該系統將管理所有剩餘的 4 KB 頁面框架。

This startup allocator is used only during early boot (before Buddy System is ready).

此啟動分配器僅在早期開機期間使用（Buddy System 尚未準備好）。

Note 註

"Mark reserved regions" is to detect the reserved regions and prevent the startup allocator from allocating them.

「標記保留區域」是用來偵測這些保留區域，並阻止啟動分配者分配它們。

Todo 全部

Implement the startup allocation.

實施啟動分配。

Note for this lab

本實驗室備註

- A general kernel should handle total physical memory and any holes reported by the Device Tree. This lab only cares about the first memory region for simplicity. But you are also encouraged to handle all usable memory regions (and be careful about the holes and the index calculation).

  一般核心應該會處理全部實體記憶體以及裝置樹報告的任何漏洞。這個實驗室只在意第一個記憶區域，為了簡化。但你也被鼓勵處理所有可用的記憶體區域（並且要注意洞穴和索引計算）。

- *The Frame Array* must be allocated **dynamically** via the Startup Allocator.

  框架陣列必須透過啟動配置器動態分配。

- Parsing the Device Tree to detect reserved regions is not part of the Startup Allocator itself. Instead, it merely calls the API designed in the previous section to mark those regions as reserved.

  解析裝置樹以偵測保留區域並非啟動分配器本身的一部分。它只是呼叫前一節設計的 API，將這些區域標記為保留區域。

- Do not hardcode any physical addresses. All memory ranges (usable or reserved) must be obtained from the OrangePi RV2 Device Tree or linker symbols:

  不要硬編碼任何實體地址。所有記憶體範圍（可用或保留）必須從 OrangePi RV2 裝置樹或連結符號中取得：

  1. Kernel image region 核心映像區域
  2. Initramfs region 伊尼特拉姆夫地區
  3. Device Tree blob itself 裝置樹 blob 本身
  4. Any additional platform-specific reserved areas (OpenSBI, Framebuffer, etc.)

     任何額外的平台專用保留區域（OpenSBI、Framebuffer 等）